DAY 7 | POST-READ

Advanced FastAPI, Security and MongoDB: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Depends()	Declare a function as a dependency; yield for setup and clean up; override in tests.
Middleware and errors	Wrap every request for timing and ids; exception handlers return clean JSON.
Authn vs authz	Who you are (401) vs what you may do (403).
Passwords	Slow, salted hashes (bcrypt, argon2, pbkdf2). Never plain text.
JWT (NEW v4: scopes)	Signed, readable, short-lived (exp), with scopes checked per route.
OWASP API Top 10 (NEW v4)	API1 object authorisation, API3 property authorisation, API4 resource limits...
MongoDB APIs	Validate ObjectId; aggregation with $match, $group, $sort, $project; skip and limit.
NoSQL injection (NEW v4)	Typed Pydantic input; escape regex; never pass raw JSON into a query.
bandit and pip-audit (NEW v4)	Scan source and dependencies; log findings in the vulnerability report.

Part B: Going beyond today

1. Overriding dependencies in tests

```
app.dependency_overrides[current_user] = lambda: {"sub": "tester", "role": "admin",
                                                  "scopes": ["tickets:read"]}
```

2. Access and refresh tokens

Short-lived access tokens (15 minutes) limit damage if stolen. A longer-lived refresh token, stored more carefully and revocable, gets a new access token without logging in again.

3. Secrets belong in the environment

Load SECRET_KEY and MONGO_URI from environment variables or a git-ignored .env file. bandit flags hard-coded secrets as B105.

4. Rate limiting

Protect /token from password guessing by limiting attempts per user or IP (OWASP API4). Libraries such as slowapi plug into FastAPI.

5. Indexes

Index the fields you filter and sort on most (status, priority, owner). Faster reads, slightly slower writes.

6. Scanning in CI

On Day 20 the same bandit and pip-audit commands run in your pipeline as security gates: the build fails until findings are fixed. That is vulnerability report v2.

Part C: Check yourself

- What runs after the yield in a dependency, and when?
- Why must password hashing be slow?
- Which OWASP API risk is "changing the id in the URL shows another user's data"?
- How does {"$ne": ""} bypass a password check, and how does Pydantic stop it?
- What is the difference between what bandit and pip-audit check?

Part D: Practice ideas for tonight

- Add a require_scope("tickets:write") dependency to every write route.
- Try three injection payloads against your search and login endpoints and record the results.
- Ask Codex to fix one bandit finding, then re-scan and mark it verified in your report.

Coming up on Day 8: JavaScript fundamentals and async JavaScript.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today